

## Mastering TensorFlow: Day 3 - Tensor Operations, Slicing, and Indexing

### Recap of Day 2

Yesterday, we explored the basics of `tf.constant` and `tf.Variable`. Today, we'll build on that knowledge by performing operations on tensors, including slicing and indexing.

```
import tensorflow as tf
```

```
# Creating tensors for operations
a = tf.constant([10, 20, 30, 40])
b = tf.constant([1, 2, 3, 4])

# Element-wise addition
add_result = tf.add(a, b)
print("Element-wise Addition:", add_result)

# Element-wise subtraction
sub_result = tf.subtract(a, b)
print("Element-wise Subtraction:", sub_result)

# Element-wise multiplication
mul_result = tf.multiply(a, b)
print("Element-wise Multiplication:", mul_result)

# Element-wise division
div_result = tf.divide(a, b)
print("Element-wise Division:", div_result)
```

```
➦ Element-wise Addition: tf.Tensor([11 22 33 44], shape=(4,), dtype=int32)
  Element-wise Subtraction: tf.Tensor([ 9 18 27 36], shape=(4,), dtype=int32)
  Element-wise Multiplication: tf.Tensor([ 10  40  90 160], shape=(4,), dtype=int32)
  Element-wise Division: tf.Tensor([10. 10. 10. 10.], shape=(4,), dtype=float64)
```

```
# Creating a 2D tensor
tensor_2d = tf.constant([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print("Original 2D Tensor:\n", tensor_2d)

# Slicing to extract the first two rows and columns
```

```
sliced_tensor = tensor_2d[:2, :2]
print("Sliced Tensor:\n", sliced_tensor)

# Indexing to extract a single element
indexed_tensor = tensor_2d[1, 2]
print("Indexed Element:", indexed_tensor)
```

 Original 2D Tensor:

```
tf.Tensor(
[[1 2 3]
 [4 5 6]
 [7 8 9]], shape=(3, 3), dtype=int32)
Sliced Tensor:
tf.Tensor(
[[1 2]
 [4 5]], shape=(2, 2), dtype=int32)
Indexed Element: tf.Tensor(6, shape=(), dtype=int32)
```

```
# Transposing a tensor
transposed_tensor = tf.transpose(tensor_2d)
print("Transposed Tensor:\n", transposed_tensor)

# Reshaping a tensor
reshaped_tensor = tf.reshape(tensor_2d, [1, 9])
print("Reshaped Tensor:\n", reshaped_tensor)
```

 Transposed Tensor:

```
tf.Tensor(
[[1 4 7]
 [2 5 8]
 [3 6 9]], shape=(3, 3), dtype=int32)
Reshaped Tensor:
tf.Tensor([[1 2 3 4 5 6 7 8 9]], shape=(1, 9), dtype=int32)
```

```
# Creating a tensor with specific data type
float_tensor = tf.constant([1.2, 3.4, 5.6], dtype=tf.float32)
print("Float Tensor:", float_tensor)
print("Data type:", float_tensor.dtype)

# Checking the shape of a tensor
tensor_shape = reshaped_tensor.shape
print("Shape of the Tensor:", tensor_shape)
```

 Float Tensor: tf.Tensor([1.2 3.4 5.6], shape=(3,), dtype=float32)

```
Data type: <dtype: 'float32'>
Shape of the Tensor: (1, 9)
```

```
# Converting data types of tensors
int_tensor = tf.cast(float_tensor, dtype=tf.int32)
```

```
print("Converted to Integer Tensor:", int_tensor)
```

```
Converted to Integer Tensor: tf.Tensor([1 3 5], shape=(3,), dtype=int32)
```

## ✓ Conclusion

Today, we explored essential tensor operations, slicing, indexing, and advanced manipulations like transposing and reshaping. These operations are foundational for working with data in TensorFlow and will be crucial as we move forward.

Tomorrow, we'll start working with real datasets in TensorFlow. Stay tuned for more! 🚀

Start coding or [generate](#) with AI.